— Entrega 2. Natalia Puñal

Sistema de Gestión de Tienda en Línea (E-Commerce)

ÍNDICE

1. Introducción
2. Objetivo
3. Situación Problemática
4. Modelo de Negocio
5. Diagrama Entidad-Relación
6. Listado de Tablas
7. Vistas
8. Funciones Personalizadas
9. Stored Procedures
10. Triggers
11. Scripts SQL

---

1. INTRODUCCIÓN

El presente proyecto consiste en el diseño e implementación de una base de datos relacional para un sistema de **tienda en línea (e-commerce)**. Desarrollada en **MySQL 8.0**, la base de datos gestiona el ciclo completo de venta digital: registro de productos y clientes, procesamiento de pedidos, control de pagos y seguimiento logístico de envíos.

En esta segunda entrega se amplía la solución incorporando **Vistas**, **Funciones Personalizadas**, **Stored Procedures** y **Triggers**, elementos que dotan al sistema de lógica de negocio embebida, automatización de procesos y capacidades analíticas avanzadas directamente desde la capa de datos.

---

2. OBJETIVO

Diseñar e implementar una base de datos relacional normalizada que soporte las operaciones de una tienda en línea, extendida con objetos de base de datos que automatizan procesos y centralizan lógica de negocio.

🛒 Comercial

#	Problemática	Consecuencia
1	Registro manual de pedidos	Duplicación y pérdida de datos
2	Sin control de stock en tiempo real	Ventas sin existencia disponible
3	Pagos no vinculados a pedidos	Errores contables
4	Sin historial de direcciones	Errores frecuentes en envíos
5	Sin seguimiento de envíos	Mala experiencia del cliente
6	Precios no registrados al comprar	Inconsistencias en facturación
7	Lógica de negocio en aplicación	Inconsistencia entre sistemas clientes
8	Sin auditoría de cambios en stock	Imposible rastrear pérdidas de inventario

- Gestionar catálogo de productos por categoría
- Registrar pedidos con detalle histórico de precios

📦 Logística

- Controlar stock con alertas de mínimo
- Registrar y seguir el estado de cada envío

💳 Contable / Financiera

- Registrar pagos con referencia y estado
- Soportar múltiples métodos de pago

📊 Analítica

- Exponer vistas consolidadas de ventas, clientes y productos
- Proveer funciones reutilizables de cálculo financiero

⚙️ Automatización

- Triggers que mantienen integridad y auditoría automáticamente
- Stored Procedures que encapsulan operaciones complejas de negocio

---

---

3. SITUACIÓN PROBLEMÁTICA

Una empresa de comercio electrónico administra simultáneamente inventario, clientes, ventas, pagos y entregas. Sin una estructura centralizada, estos procesos generan inconsistencias, pérdida de información y errores operativos.

Problemáticas Identificadas

Brechas que Soluciona la Base de Datos

- ✅ Centralización de información en sistema relacional único
- ✅ Integridad referencial mediante claves foráneas
- ✅ Trazabilidad completa: pedido → pago → envío
- ✅ Control de inventario automático vía Triggers
- ✅ Lógica de negocio encapsulada en Stored Procedures
- ✅ Consultas analíticas pre construidas mediante Vistas
- ✅ Cálculos reutilizables mediante Funciones Personalizadas

---

4. MODELO DE NEGOCIO

Descripción de la Organización

Empresa de comercio electrónico B2C que opera exclusivamente en plataforma digital, comercializando productos físicos organizados por categorías.

Flujo Operativo

```
┌──────────────────────────────────────────────────────────────────┐
│                      TIENDA EN LÍNEA                             │
│                                                                  │
│  [CLIENTE]──→[CATÁLOGO]──→[CARRITO]──→[PEDIDO]                  │
│                                          │                       │
│                                  ┌───────┴────────┐              │
│                                  ▼                ▼              │
│                              [PAGO]           [ENVÍO]            │
│                                  │                │              │
│                      [TRIGGER: auditoría]  [TRIGGER: stock]      │
│                                  ▼                ▼              │
│                          [CONFIRMADO]       [ENTREGADO]          │
└──────────────────────────────────────────────────────────────────┘
```

Actores del Sistema

Actor	Rol
Cliente	Registra cuenta, realiza pedidos y pagos
Producto	Unidad comercializable con precio, categoría e inventario
Administrador	Gestiona catálogo, stock y seguimiento
Transportista	Ejecuta el envío físico registrado en el sistema

Procesos Clave

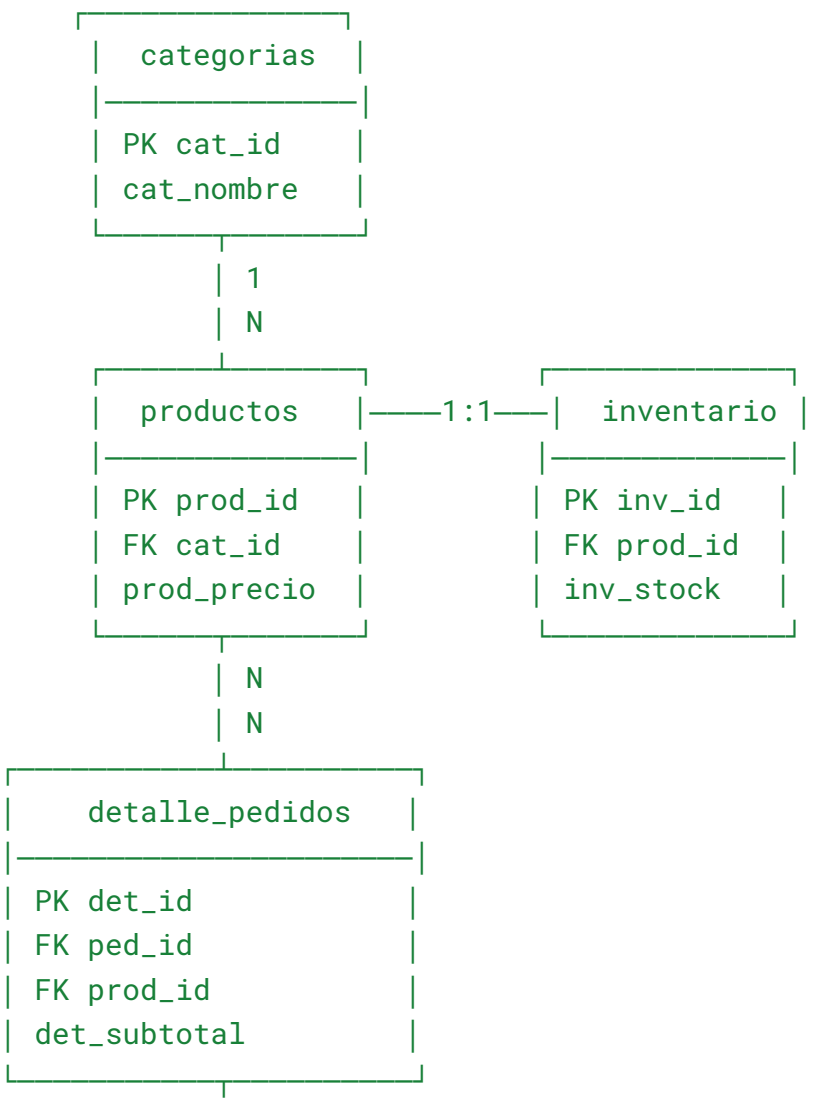
```
PROCESO 1 — Alta de Producto
categorias → productos → inventario
```

```
PROCESO 2 — Compra de Cliente
clientes → direcciones → pedidos → detalle_pedidos
          [SP: sp_crear_pedido] [TRIGGER: trg_actualizar_stock]

PROCESO 3 — Cobro
pedidos → pagos → metodos_pago
         [TRIGGER: trg_actualizar_estado_pedido]

PROCESO 4 — Entrega
pedidos → envios
         [SP: sp_registrar_envio]
```

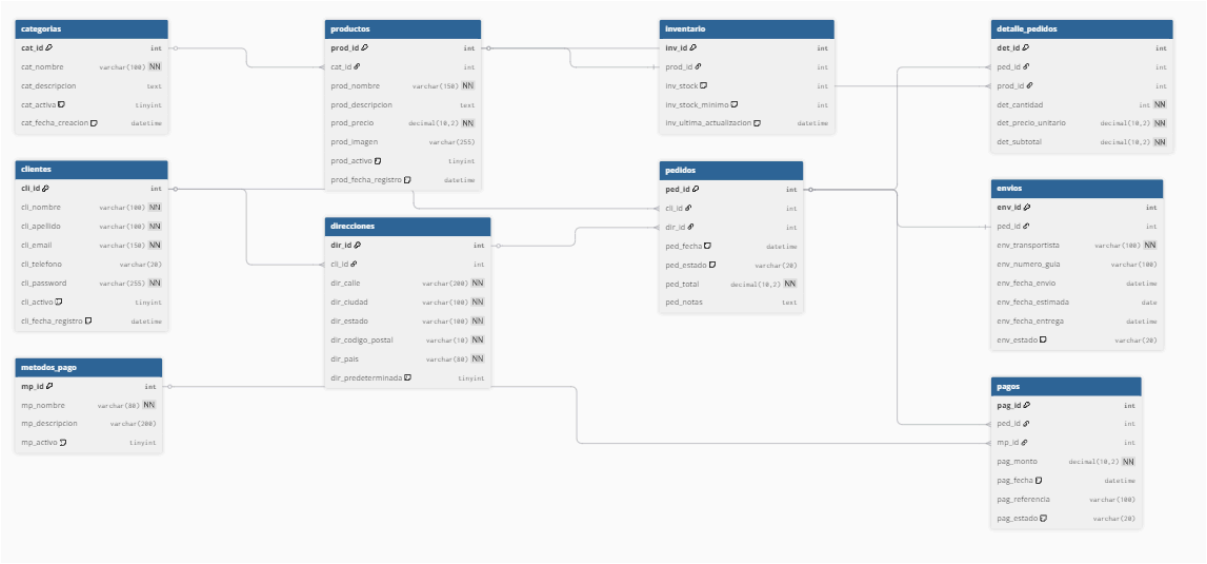
---

5. DIAGRAMA ENTIDAD-RELACIÓN

📌 Diagrama generado con **dbdiagram.io**. Representa las 10 entidades y sus relaciones de cardinalidad.



```
                                  | N
                                  | 1
┌─────────────────┐     ┌─────────┴─────────┐     ┌─────────────────┐
│   direcciones   │     │     pedidos       │     │     envios      │
├─────────────────┤     ├───────────────────┤     ├─────────────────┤
│ PK dir_id       │N─1─│ PK ped_id         │1─1─│ PK env_id       │
│ FK cli_id       │     │ FK cli_id         │     │ FK ped_id       │
│ dir_ciudad      │     │ ped_estado        │     │ env_estado      │
└─────────────────┘     │ ped_total         │     └─────────────────┘
                        └─────────┬─────────┘
                                  | 1
                                  | N
                        ┌─────────┴─────────┐     ┌─────────────────────┐
                        │      pagos        │N─1─│   metodos_pago      │
                        ├───────────────────┤     ├─────────────────────┤
                        │ PK pag_id         │     │ PK mp_id            │
                        │ FK ped_id         │     │ mp_nombre           │
                        │ pag_monto         │     └─────────────────────┘
                        └───────────────────┘

┌─────────────────┐
│    clientes     │──1:N──→ direcciones
├─────────────────┤──1:N──→ pedidos
│ PK cli_id       │
│ cli_email       │
└─────────────────┘
```



Relaciones y Cardinalidad

Entidad Origen	Cardinalidad	Entidad Destino
categorías	1 : N	productos
productos	1 : 1	inventario
clientes	1 : N	direcciones
clientes	1 : N	pedidos
direcciones	1 : N	pedidos
pedidos	1 : N	detalle_pedidos
productos	1 : N	detalle_pedidos
pedidos	1 : 1	envios
pedidos	1 : N	pagos
metodos_pago	1 : N	pagos

---

6. LISTADO DE TABLAS

Tabla 1: `categorías`

Descripción: Clasifica los productos en grupos temáticos.

Abreviatura	Nombre Completo	Tipo de Dato	Clave
`cat_id`	ID Categoría	INT UNSIGNED	**PK** AUTO_INCREMENT
`cat_nombre`	Nombre Categoría	VARCHAR(100)	NOT NULL
`cat_descripcion`	Descripción	TEXT	NULL
`cat_activa`	Activa	TINYINT(1)	DEFAULT 1
`cat_fecha_creacion`	Fecha Creación	DATETIME	DEFAULT NOW()

Tabla 2: `productos`

Descripción: Catálogo completo de productos disponibles en la tienda.

Abreviatura	Nombre Completo	Tipo de Dato	Clave
`prod_id`	ID Producto	INT UNSIGNED	**PK** AUTO_INCREMENT
`cat_id`	ID Categoría	INT UNSIGNED	**FK** → categorías
`prod_nombre`	Nombre	VARCHAR(150)	NOT NULL — **IDX**
`prod_descripcion`	Descripción	TEXT	NULL
`prod_precio`	Precio Unitario	DECIMAL(10,2)	NOT NULL
`prod_imagen`	URL Imagen	VARCHAR(255)	NULL
`prod_activo`	Activo	TINYINT(1)	DEFAULT 1
`prod_fecha_registro`	Fecha Registro	DATETIME	DEFAULT NOW()

Índices: `idx_prod_categoria`, `idx_prod_nombre`

Tabla 3: `inventario`

Descripción: Controla el stock por producto. Actualizado automáticamente por Triggers.

Abreviatura	Nombre Completo	Tipo de Dato	Clave
`inv_id`	ID Inventario	INT UNSIGNED	**PK** AUTO_INCREMENT
`prod_id`	ID Producto	INT UNSIGNED	**FK** → productos — **UNIQUE**
`inv_stock`	Stock Disponible	INT UNSIGNED	DEFAULT 0
`inv_stock_minimo`	Stock Mínimo	INT UNSIGNED	DEFAULT 5
`inv_ultima_actualizacion`	Última Actualización	DATETIME	ON UPDATE NOW()

Tabla 4: `clientes`

Descripción: Información personal y de acceso de clientes registrados.

Abreviatura	Nombre Completo	Tipo de Dato	Clave
`cli_id`	ID Cliente	INT UNSIGNED	**PK** AUTO_INCREMENT
`cli_nombre`	Nombre	VARCHAR(100)	NOT NULL
`cli_apellido`	Apellido	VARCHAR(100)	NOT NULL
`cli_email`	Correo Electrónico	VARCHAR(150)	**UNIQUE — IDX**
`cli_telefono`	Teléfono	VARCHAR(20)	NULL
`cli_password`	Contraseña Hash	VARCHAR(255)	NOT NULL
`cli_activo`	Activo	TINYINT(1)	DEFAULT 1
`cli_fecha_registro`	Fecha Registro	DATETIME	DEFAULT NOW()

Tabla 5: `direcciones`

Descripción: Direcciones de envío por cliente, con opción de predeterminada.

Abreviatura	Nombre Completo	Tipo de Dato	Clave
`dir_id`	ID Dirección	INT UNSIGNED	**PK** AUTO_INCREMENT
`cli_id`	ID Cliente	INT UNSIGNED	**FK** → clientes — **IDX**
`dir_calle`	Calle y Número	VARCHAR(200)	NOT NULL
`dir_ciudad`	Ciudad	VARCHAR(100)	NOT NULL
`dir_estado`	Estado / Provincia	VARCHAR(100)	NOT NULL
`dir_codigo_postal`	Código Postal	VARCHAR(10)	NOT NULL
`dir_pais`	País	VARCHAR(80)	NOT NULL
`dir_predeterminada`	Predeterminada	TINYINT(1)	DEFAULT 0

Tabla 6: `pedidos`

Descripción: Cabecera de cada orden de compra con estado y total.

Abreviatura	Nombre Completo	Tipo de Dato	Clave
`ped_id`	ID Pedido	INT UNSIGNED	**PK** AUTO_INCREMENT
`cli_id`	ID Cliente	INT UNSIGNED	**FK** → clientes — **IDX**
`dir_id`	ID Dirección	INT UNSIGNED	**FK** → direcciones
`ped_fecha`	Fecha Pedido	DATETIME	DEFAULT NOW() — **IDX**
`ped_estado`	Estado	ENUM	pendiente/procesando/enviado/entregado/cancelado — **IDX**
`ped_total`	Total	DECIMAL(10, 2)	NOT NULL
`ped_notas`	Notas	TEXT	NULL

Índices: `idx_ped_cliente`, `idx_ped_estado`, `idx_ped_fecha`

Tabla 7: `detalle_pedidos`

Descripción: Productos por pedido con precio histórico al momento de compra.

Abreviatura	Nombre Completo	Tipo de Dato	Clave
`det_id`	ID Detalle	INT UNSIGNED	**PK** AUTO_INCREMENT
`ped_id`	ID Pedido	INT UNSIGNED	**FK** → pedidos — **IDX**
`prod_id`	ID Producto	INT UNSIGNED	**FK** → productos — **IDX**
`det_cantidad`	Cantidad	INT UNSIGNED	NOT NULL
`det_precio_unitario`	Precio Unitario	DECIMAL(10,2)	NOT NULL
`det_subtotal`	Subtotal	DECIMAL(10,2)	NOT NULL

Tabla 8: `metodos_pago`

Descripción: Catálogo de métodos de pago habilitados en la plataforma.

Abreviatura	Nombre Completo	Tipo de Dato	Clave

`mp_id`	ID Método Pago	INT UNSIGNED	**PK** AUTO_INCREMENT
`mp_nombre`	Nombre	VARCHAR(80)	NOT NULL
`mp_descripcion`	Descripción	VARCHAR(200)	NULL
`mp_activo`	Activo	TINYINT(1)	DEFAULT 1

Tabla 9: `pagos`

Descripción: Pagos realizados por pedido con estado y referencia de transacción.

Abreviatura	Nombre Completo	Tipo de Dato	Clave
`pag_id`	ID Pago	INT UNSIGNED	**PK** AUTO_INCREMENT
`ped_id`	ID Pedido	INT UNSIGNED	**FK** → pedidos — **IDX**
`mp_id`	ID Método Pago	INT UNSIGNED	**FK** → metodos_pago
`pag_monto`	Monto	DECIMAL(10,2)	NOT NULL
`pag_fecha`	Fecha Pago	DATETIME	DEFAULT NOW()
`pag_referencia`	Referencia	VARCHAR(100)	NULL
`pag_estado`	Estado	ENUM	pendiente/completado/fallido/reembolsado — **IDX**

Tabla 10: `envios`

Descripción: Seguimiento logístico del envío físico de cada pedido.

Abreviatura	Nombre Completo	Tipo de Dato	Clave
`env_id`	ID Envío	INT UNSIGNED	**PK** AUTO_INCREMENT

`ped_id`	ID Pedido	INT UNSIGNED	**FK** → pedidos — **UNIQUE**
`env_transportista`	Transportista	VARCHAR(100)	NOT NULL
`env_numero_guia`	Número de Guía	VARCHAR(100)	NULL
`env_fecha_envio`	Fecha Envío	DATETIME	NULL
`env_fecha_estimada`	Fecha Estimada	DATE	NULL
`env_fecha_entrega`	Fecha Entrega Real	DATETIME	NULL
`env_estado`	Estado	ENUM	preparando/en_camino/entregado/devuelto — **IDX**

Tabla 11: `auditoria_stock`

Descripción: Registra automáticamente cada movimiento de inventario generado por Triggers, permitiendo trazabilidad de cambios de stock.

Abreviatura	Nombre Completo	Tipo de Dato	Clave
`aud_id`	ID Auditoría	INT UNSIGNED	**PK** AUTO_INCREMENT
`prod_id`	ID Producto	INT UNSIGNED	**FK** → productos — **IDX**
`aud_stock_anterior`	Stock Anterior	INT	NOT NULL
`aud_stock_nuevo`	Stock Nuevo	INT	NOT NULL
`aud_motivo`	Motivo del Cambio	VARCHAR(100)	NOT NULL
`aud_fecha`	Fecha Registro	DATETIME	DEFAULT NOW()

---

7. VISTAS

Vista 1: `vw_productos_catalogo`

Descripción: Muestra el catálogo de productos activos con su categoría y stock disponible. **Objetivo:** Facilitar la consulta del catálogo completo desde cualquier sistema cliente sin necesidad de construir el JOIN manualmente. **Tablas que la componen:** `productos`, `categorias`, `inventario`

Campo Expuesto	Origen
prod_id	productos
prod_nombre	productos
prod_precio	productos
cat_nombre	categorías
inv_stock	inventario

Vista 2: `vw_pedidos_detalle`

Descripción: Consolida cada pedido con los datos del cliente, dirección de envío, total y estado actual. **Objetivo:** Proveer una vista operativa completa de pedidos para reportes de gestión y seguimiento. **Tablas que la componen:** `pedidos`, `clientes`, `direcciones`

Campo Expuesto	Origen
ped_id	pedidos
ped_fecha	pedidos
ped_estado	pedidos
ped_total	pedidos
cliente_nombre	clientes
cliente_email	clientes
dir_ciudad	direcciones
dir_pais	direcciones

Vista 3: `vw_ventas_por_producto`

Descripción: Agrega el total de unidades vendidas y el ingreso generado por cada producto. **Objetivo:** Soporte analítico para identificar los productos de mayor rotación e ingresos. **Tablas que la componen:** `detalle_pedidos`, `productos`

Campo Expuesto	Origen
prod_id	productos
prod_nombre	productos
total_unidades_vendidas	SUM(det_cantidad)
ingreso_total	SUM(det_subtotal)

Vista 4: `vw_pagos_completados`

Descripción: Lista todos los pagos con estado 'completado', incluyendo método utilizado y pedido asociado. **Objetivo:** Uso contable para conciliación de pagos recibidos. **Tablas que la componen:** `pagos`, `pedidos`, `metodos_pago`

Campo Expuesto	Origen
pag_id	pagos
ped_id	pedidos
pag_monto	pagos
pag_fecha	pagos
pag_referencia	pagos
mp_nombre	metodos_pago

Vista 5: `vw_stock_bajo`

Descripción: Muestra los productos cuyo stock actual es igual o inferior al stock mínimo definido. **Objetivo:** Alertar al área de logística sobre productos que requieren reabastecimiento urgente. **Tablas que la componen:** `inventario`, `productos`

Campo Expuesto	Origen
prod_id	productos
prod_nombre	productos
inv_stock	inventario
inv_stock_minimo	inventario
diferencia	inv_stock - inv_stock_minimo

---

8. FUNCIONES PERSONALIZADAS

Función 1: `fn_calcular_total_pedido`

Descripción: Calcula el monto total de un pedido sumando los subtotales de su detalle. **Objetivo:** Centralizar el cálculo del total del pedido en la base de datos, evitando discrepancias entre sistemas. **Tablas/datos que utiliza:** `detalle_pedidos` **Parámetro de entrada:** `p_ped_id INT UNSIGNED` **Retorna:** `DECIMAL(10,2)`

Función 2: `fn_total_compras_cliente`

Descripción: Retorna el monto total histórico gastado por un cliente en pedidos con pago completado. **Objetivo:** Soporte para segmentación de clientes por valor de compra acumulado. **Tablas que utiliza:** `pedidos`, `pagos` **Parámetro de entrada:** `p_cli_id INT UNSIGNED` **Retorna:** `DECIMAL(10,2)`

Función 3: `fn_stock_disponible`

Descripción: Retorna el stock actual de un producto específico. **Objetivo:** Consulta rápida y reutilizable de disponibilidad para validaciones previas a la venta. **Tablas que utiliza:** `inventario` **Parámetro de entrada:** `p_prod_id INT UNSIGNED` **Retorna:** `INT UNSIGNED`

Función 4: `fn_nombre_completo_cliente`

Descripción: Concatena nombre y apellido del cliente en un único campo formateado. **Objetivo:** Estandarizar la presentación del nombre completo en vistas y reportes. **Tablas que utiliza:** `clientes` **Parámetro de entrada:** `p_cli_id INT UNSIGNED` **Retorna:** `VARCHAR(205)`

---

9. STORED PROCEDURES

SP 1: `sp_crear_pedido`

Descripción: Crea un pedido completo con su detalle. Válida stock disponible antes de insertar y calcula el total automáticamente. **Objetivo:** Encapsular la lógica de creación de pedido en una sola transacción atómica. **Tablas que impacta:** `pedidos`, `detalle_pedidos`, `inventario` **Parámetros:**

- `p_cli_id` — ID del cliente
- `p_dir_id` — ID de la dirección
- `p_prod_id` — ID del producto
- `p_cantidad` — Cantidad a comprar

SP 2: `sp_registrar_pago`

Descripción: Registra un pago para un pedido y actualiza el estado del pedido a 'procesando' si el pago es completado. **Objetivo:** Garantizar consistencia entre el registro del pago y el estado del pedido en una sola operación. **Tablas que impacta:** `pagos`, `pedidos` **Parámetros:**

- `p_ped_id` — ID del pedido
- `p_mp_id` — ID del método de pago
- `p_monto` — Monto del pago
- `p_referencia` — Referencia o folio

SP 3: `sp_registrar_envio`

Descripción: Crea el registro de envío de un pedido y actualiza el estado del pedido a 'enviado'. **Objetivo:** Automatizar el cambio de estado del pedido al iniciar el proceso logístico. **Tablas que impacta:** `envíos`, `pedidos` **Parámetros:**

- `p_ped_id` — ID del pedido
- `p_transportista` — Nombre del transportista
- `p_guia` — Número de guía
- `p_fecha_estimada` — Fecha estimada de entrega

SP 4: `sp_reporte_ventas_periodo`

Descripción: Genera un reporte de ventas totales por producto dentro de un rango de fechas. **Objetivo:** Proveer información analítica de ventas por período para toma de decisiones comerciales. **Tablas que utiliza:** `pedidos`, `detalle_pedidos`, `productos` **Parámetros:**

- `p_fecha_inicio` — Fecha de inicio del período
- `p_fecha_fin` — Fecha de fin del período

---

10. TRIGGERS

Trigger 1: `trg_descontar_stock`

Descripción: Al insertar un registro en `detalle_pedidos`, descuenta automáticamente la cantidad vendida del stock en `inventario`. **Objetivo:** Mantener el inventario actualizado en tiempo real sin intervención manual. **Tabla origen:** `detalle_pedidos` **Evento:** `AFTER INSERT` **Tabla impactada:** `inventario`

Trigger 2: `trg_auditoria_stock`

Descripción: Al actualizar el stock en `inventario`, registra el valor anterior y el nuevo en la tabla `auditoria_stock` con fecha y motivo. **Objetivo:** Mantener trazabilidad completa de todos los movimientos de inventario. **Tabla origen:** `inventario` **Evento:** `AFTER UPDATE` **Tabla impactada:** `auditoria_stock`

Trigger 3: `trg_estado_entregado`

Descripción: Al actualizar `envíos` con estado 'entregado', actualiza automáticamente el estado del pedido asociado a 'entregado' y registra la fecha de entrega real. **Objetivo:** Sincronizar el estado del pedido con el estado logístico del envío sin intervención manual. **Tabla origen:** `envios` **Evento:** `AFTER UPDATE` **Tabla impactada:** `pedidos`

Trigger 4: `trg_validar_stock_minimo`

Descripción: Antes de actualizar el stock en `inventario`, verifica que no quede en valor negativo. Si el nuevo stock es negativo, lanza un error controlado. **Objetivo:** Evitar inconsistencias de stock negativo por ventas concurrentes o errores de carga. **Tabla origen:** `inventario` **Evento:** `BEFORE UPDATE` **Tabla impactada:** `inventario` (protección)

---

11. SCRIPTS SQL

Archivo 1: `tienda_online_creacion.sql`

https://github.com/nathalia2000-web/Entrega2-Pu-al/blob/main/tienda_online_creacion.sql

Creación de base de datos, tablas, vistas, funciones, stored procedures y triggers.

Archivo 2: `tienda_online_datos_prueba.sql`

https://github.com/nathalia2000-web/Entrega2-Pu-al/blob/main/tienda_online_datos_prueba.sql

Inserción de datos de prueba y ejecución de stored procedures para validar el funcionamiento de todos los objetos.